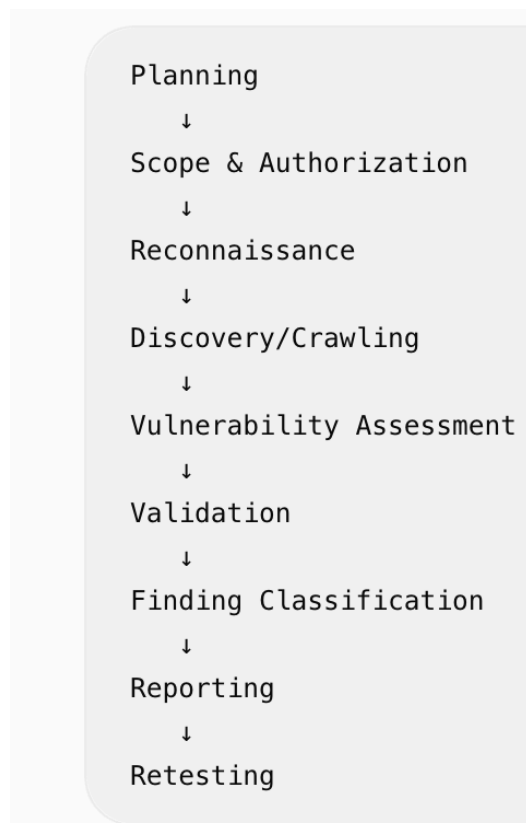


1. Methodology

The Automated Web Penetration Testing Toolkit follows the Penetration Testing Execution Standard (PTES) as its primary penetration-testing methodology. The methodology provides a structured process for planning, conducting, documenting, and reporting an authorized security assessment. The project adapts the methodology to a controlled web-application testing environment containing intentionally vulnerable applications. OWASP guidance is used to identify appropriate web-application security testing areas.



2. Scope

Inscope

The scope includes two intentionally vulnerable web applications to test against: OWASP Juice Shop, and DVWA, both hosted in a docker environment.

Activities:

- Target identification
- HTTP reconnaissance
- Website crawling
- Endpoint discovery
- Form and parameter discovery
- Security-header checks
- Controlled vulnerability testing
- Finding classification
- Evidence collection
- Reporting
- Retesting/validation

Out of Scope

- Public websites
- Production systems
- Systems belonging to other individuals or organizations
- Personal accounts
- Denial-of-service testing
- Destructive testing
- Data destruction
- Malware deployment outside the controlled lab
- Credential attacks against unauthorized accounts

3. Objectives

Primary Objective

Develop and demonstrate an automated web penetration-testing toolkit capable of performing repeatable security testing against intentionally vulnerable web applications in a controlled environment.

Supporting Objectives

1. Identify accessible pages, endpoints, forms, and parameters.

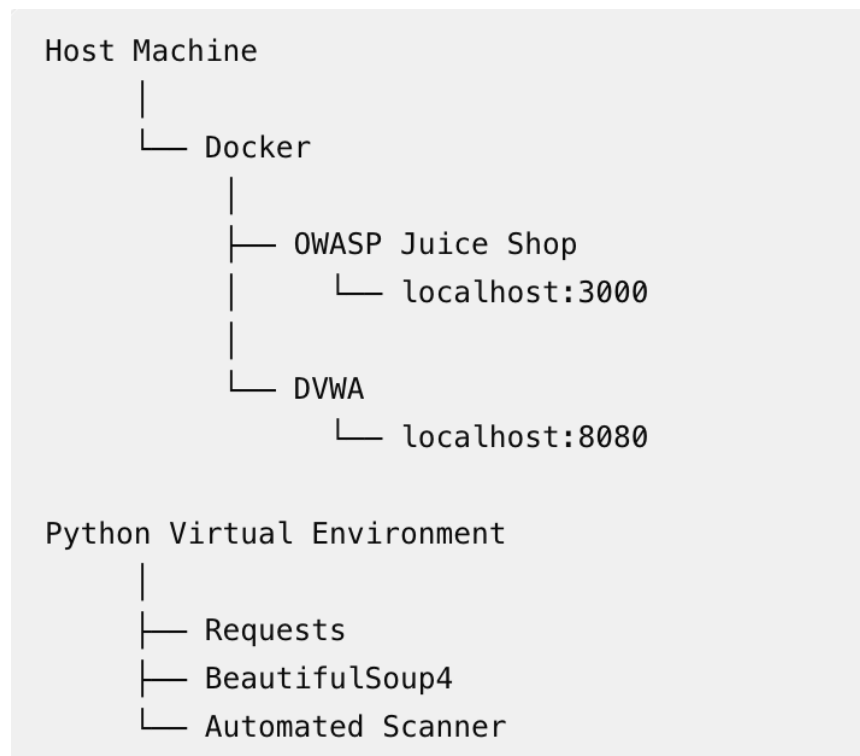
2. Perform selected web-security checks.
3. Record evidence associated with security findings.
4. Categorize findings according to severity.
5. Produce an HTML penetration-testing report.
6. Demonstrate the methodology through a sample engagement.

4. Rules of Engagement

Rule	Requirement
Authorization	Testing must be authorized before scanning begins
Targets	Only approved lab targets may be scanned
Environment	Testing is performed against controlled Docker applications
Testing hours	Testing occurs during the project's scheduled development/testing period
Data handling	Do not intentionally collect unnecessary sensitive information
Destructive testing	Not permitted

DoS testing	Not permitted
Target changes	A new target requires updated authorization/scope
Unexpected impact	Stop testing if unexpected system impact occurs
Evidence	Save relevant logs, screenshots, requests/responses, and findings

5. Testing Environment



Tool	Purpose
Python	Scanner development
Requests	HTTP communication
BeautifulSoup	HTML parsing/crawling
Docker	Controlled testing environment
Juice Shop	Vulnerable web application
DVWA	Vulnerable web application
Git	Version control
Nmap	Network/service discovery where applicable
OWASP guidance	Web security testing reference
PTES	Overall penetration-testing methodology

6. Testing Boundaries

Allowed:

localhost:3000 → Juice Shop
localhost:8080 → DVWA

Allowed activities:

Reconnaissance
Crawling
Endpoint discovery
Parameter discovery
Controlled vulnerability checks
Evidence collection
Reporting

Not allowed:

Public websites
Production systems
Unauthorized systems
DoS testing
Destructive exploitation
Data destruction
Unapproved credential attacks

7. Timeline

Week	Methodology activity
1	Planning and requirements
2	Security research and methodology design
3	Environment configuration and test targets
4	CLI, target selection, scan settings
5	HTTP reconnaissance
6	Crawling and endpoint discovery
7	Initial vulnerability testing
8	Additional vulnerability testing and validation
9	Nuclei/ZAP integration
10	Finding management and reporting
11	Full assessment/testing
12	Final documentation, retesting, and presentation

8. Penetration Testing Procedure

Phase 1 — Authorization and Scope

Before testing:

- Confirm target.
- Confirm authorization.
- Confirm testing boundaries.

- Confirm allowed activities.
- Document the target.

Phase 2 — Reconnaissance

The toolkit identifies:

- Target URL
- HTTP status
- Response information
- Server information where available
- Accessible resources

Phase 3 — Discovery

The crawler identifies:

- Links
- Pages
- Forms
- Parameters
- Endpoints

Phase 4 — Vulnerability Testing

The toolkit performs the selected controlled checks.

Phase 5 — Finding Classification

Each finding should contain:

Finding name

Severity

Affected URL

Parameter

Description

Evidence

Potential impact

Recommended remediation

Phase 6 — Reporting

The results are organized into an HTML report.

Phase 7 — Validation/Retesting

After remediation or changes, repeat the relevant test and determine whether the finding remains.

9. Sample Engagement

Target:

Owasp juice shop (<http://localhost:3000>) or DVWA (<http://localhost:8080>)

Authorization:

Authorized project laboratory target

Methodology:

PTES-based web penetration test

Testing activities:

1. Confirm target availability
2. Perform HTTP reconnaissance
3. Crawl application
4. Identify endpoints
5. Identify forms/parameters

6. Execute selected security checks

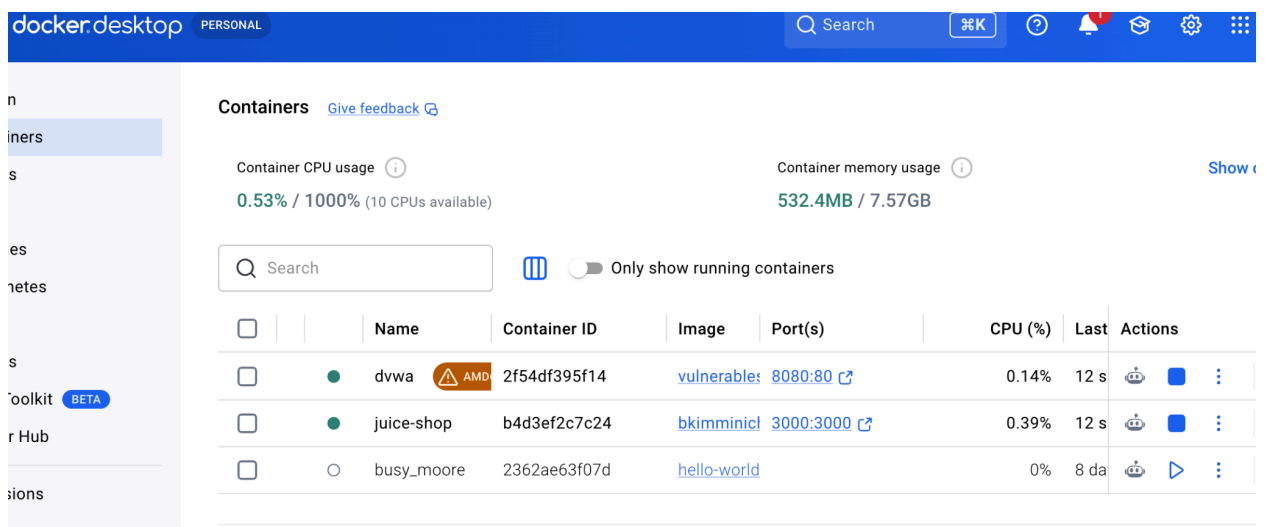
7. Record findings

8. Classify findings

9. Generate report

10. Validate results

Evidence 1: Docker containers running



The screenshot shows the Docker Desktop interface. The left sidebar contains navigation options: Home, Containers, Images, Settings, and Extensions. The main area is titled 'Containers' and shows overall usage statistics: Container CPU usage at 0.53% / 1000% (10 CPUs available) and Container memory usage at 532.4MB / 7.57GB. Below these statistics is a search bar and a toggle for 'Only show running containers'. A table lists the running containers:

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
☐	dvwa	2f54df395f14	vulnerable	8080:80	0.14%	12 s	
☐	juice-shop	b4d3ef2c7c24	bkimminic	3000:3000	0.39%	12 s	
☐	busy_moore	2362ae63f07d	hello-world		0%	8 da	

Evidence 2: Scanner CLI

```
(automated-web-pentest) DAEDMAC1010:Pentesting_Toolkit Adult$ python3 scanner.py --target http://localhost:3000 --verbose
/Users/Adult/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
=====
Automated Web Penetration Testing Framework
=====
Web Application to scan: http://localhost:3000
Request timeout: 10 seconds
Verbose mode: ON
Starting scan...
```

Evidence 3: HTTP response

```
Status Code: 200
Response Length: 9393 bytes
Content Type: text/html; charset=UTF-8
Server: None
I can reach your target Web Application! :3
```

Evidence 4: Crawler results

Not found presently

Evidence 5: Vulnerability check

Not found presently

Evidence 6: Final HTML report.

Not found presently

10. Authorization Form

PENETRATION TESTING AUTHORIZATION

Project:

Python-Based Automated Web Penetration Testing Framework

Authorized Tester:

Andrew D'Amico

Authorized Targets:

OWASP Juice Shop - localhost:3000

DVWA - localhost:8080

Purpose:

Controlled security testing and framework development.

Authorized Activities:

Reconnaissance, crawling, endpoint discovery,
controlled vulnerability testing, validation,
and reporting.

Prohibited Activities:

Denial-of-service testing, destructive testing,
unauthorized systems, production systems, and
testing outside the defined scope.

Testing Period:

August 31 - December 3 2026

Authorization:

I authorize security testing of the targets listed
above within the defined testing boundaries.

Name:

Andrew D'Amico

Signature:

Andrew D'Amico

Date:

September 17, 2026

11. Deliverables

Deliverables

1. Penetration Testing Methodology and Test Plan
2. Scope and Rules of Engagement
3. Lab Authorization Form
4. Testing Environment Documentation
5. Scanner source code
6. Test execution logs
7. Screenshots/evidence
8. Vulnerability findings
9. HTML penetration-testing report
10. Sample engagement results
11. Retesting/validation results
12. Final project documentation

